FIT3155: Week 5 Lab Sheet

(Questions are based on Week 4 topic, Ukkonen's algorithm for suffix tree construction.)

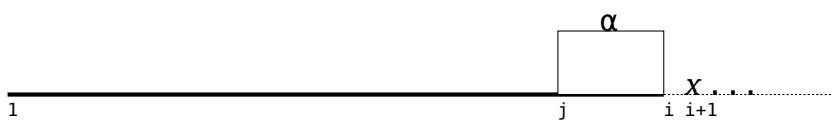
Objectives: Develop a clear understanding of Ukkonen's algorithm of a linear-time suffix tree construction and its common applications.

Conceptual exercises

1. For a string $S[1 \ldots n]$, reason the following without getting bogged down in implementational details of the Ukkonen's algorithm):

   (a) Conceptually, what information are you storing at the end of each phase of this algorithm?

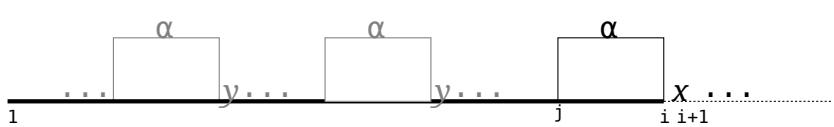
   (b) Going from some phase i to the next phase $i+1$, when extending $\alpha = S[j \ldots i]$ (i.e., the suffix j of phase i) by one more character (say) $x = S[i+1]$, reason abstractly how (or what aspects of) the implicit suffix tree changes upon encountering the following suffix-extension scenarios:

   - 

     α is encountered for the **first time** starting at position j and never at any $j' < j$.
     (This will aid your understanding of Rule 1 suffix extension.)

     As an example, handling the following string
     1 2 3 4 5 6 7 8 9 10
     a b c c b c c b d d ...,

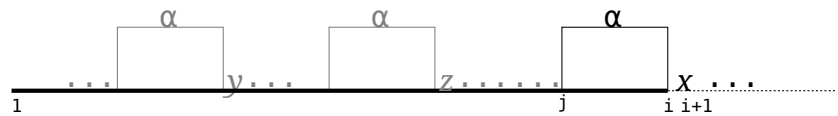
     this scenario presents itself for extension $j = 6$ in phase $i + 1 = 10$.

   - 

     α was previously observed (one or more times before j), but always followed by some same character $y \neq x$.
     (This will aid your understanding of Rule 2 (regular) suffix extension.)

     As an example, handling the following string
     1 2 3 4 5 6 7 8 9 10 11 12 13
     a b c c b c c b d d c c d ...,

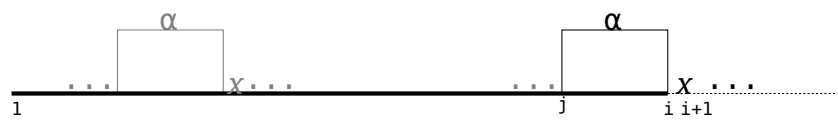
     this scenario presents itself for extension $j = 11$ in phase $i + 1 = 13$.

- 

  α was previously observed (two or more times before j), and there were at least two distinct characters following those occurrences, each **different** from x.
  (This will aid your understanding of Rule 2 (alternate) suffix extension.)

  As an example, handling the following string
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
  a b c c b c c b d d c c d c c b d c c b c c c b a ...,

  this scenario presents itself for extension $j = 22$ in phase $i + 1 = 25$.

- 

  α followed by x was previously observed at some position $j' < j$? (This will aid your understanding of Rule 3 suffix extension.)

  As an example, handling the following string
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
  a b c c b c c b d d c c d c c b d c c b c c c b a c c b d ...,

  this scenario presents itself for extension $j = 26$ in phase $i + 1 = 29$.

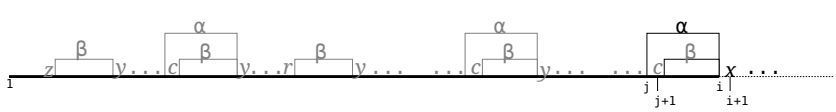
2. For a suffix extension j in phase i, suppose the applied rule is one of: Rule 1, Rule 2 (regular), Rule 2 (alternate), or Rule 3.

   For each case determine which rules are possible for:

   (a) the same extension j in phase $i + 1$;

   (b) the next extension $j + 1$ in phase i.

   Note: Reasoning this question, among other things, will lay a groundwork for understanding Tricks 3 and 4 used in Ukkonen's algorithm (Suggestion: It will help to draw abstract string diagrams like those in this lab sheet, to guide your reasoning for this question.)

3. 

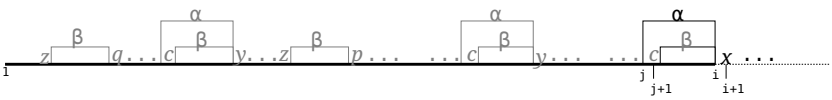
   For a string $S[1 \ldots n]$, let $\alpha = S[j \ldots i]$ and $x = S[i + 1]$. Write α as $\alpha = c\beta$, where $c = S[j]$ and $\beta = S[j + 1 \ldots i]$.

   Suppose that both substrings α and β (with β also occurring independently of α, i.e., preceded by a different character) have previously appeared in the prefix $S[1 \ldots j - 1]$, as illustrated above. (Concrete example:
   1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
   b b a b a c b a b a b a a a c b a b a a b a b a c b a a b a b c ...,

   Explore suffix extensions $j = 28$ followed by $j + 1 = 29$ in phase $i + 1 = 32$.)

   - At the start of suffix extension j in phase $i + 1$, sketch abstractly how the subtree after the path labeled α will look like. Do the same for β.

- How does this subtree change by the end of suffix extension j of phase $i+1$?
- How does it further change by the end of suffix extension $j+1$ of phase $i+1$?
- Is there a need to create a suffix link in this situation? If so, identify in your sketch the source and target node of this suffix link?

4. 

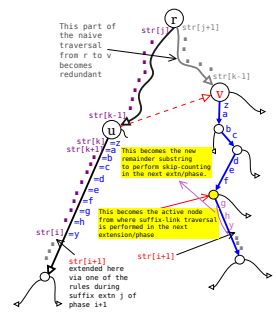
   Redo the above questions for this abstract characterization of the string.

   (Note: slightly modified example string below, compared to the one in the previous question:
   1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
   `b b a b b c b a b a b a a a c b a b d a b a b a c b a a b a b c` ...,

   Explore suffix extensions $j = 28$ followed by $j+1 = 29$ in phase $i+1 = 32$.)

5. Consider the following (abstract) illustration of the *skip-count* trick:

   Use it to understand the definitions of `Active node`, `Remainder` and `suffix link` and how they factor into the skip-counting traversal. Understand how (potentially) the active node and remainder changes after each extension or after each phase.



6. (Self-study question:) Manually trace out the mechanics of Ukkonen's algorithm (with tricks applied) for the string $S =$ `a b a a b a $` .
   1 2 3 4 5 6 7

7. (Self-study question:) Reason the worst-case space-complexity required to store the entire suffix tree of any string $S[1 \ldots n]$ derived from some alphabet Σ.

8. (Self-study questions:) Reason how the following algorithmic problems can be solved after constructing an (explicit) suffix tree of a given string $S[1 \ldots n]$:

   (a) Given a(nother) string $\alpha[1 \ldots m \leq n]$:
   - Find the number of times a string t occurs in S as a substring. What is the time complexity* of returning this number?
   - List all positions (if any) where a string α appears in S and reason its time complexity*?

   (b) Find the smallest substring(s) of S occurring exactly $k \geq 0$. What is its time complexity*?

   (c) Find the longest repeated substring within S. What is its time complexity*?

*You can assume that the suffix tree of S has been precomputed when characterizing the time complexity.

(d) Find the lexicographically smallest suffix of S. What is its time complexity*?

(e) Find the suffix array of string S. What is its time complexity*?

(f) Find the BWT of string S. What is its time complexity*?

Structured implementation of Ukkonen's algorithm

(A message of caution: 'Vibe coding' with GenAI will hinder your learning of algorithms and data structures. The degree (and this unit) you have enrolled into is a specialist degree (unit). By definition it expects one to demonstrate specialist understanding of topics. This understanding comes from making a concerted effort, via thinking, implementing, debugging etc.)

The structure below is designed to help you complete Ukkonen's algorithm implementation in a scaffolded manner:

1. Design the Node and Edge data structures that form the foundation of Ukkonen's algorithm. Ensure that your design incorporates a space-efficient representation of edge labels (refer to slide 28).

2. Implement the algorithm **without** any optimizations or speed-ups (refer to slide 17). Specifically, implement suffix extension Rules 1, 2 (both regular and alternate), and 3 by performing full suffix traversals starting from the root node.

3. Implement the 'skip/count' trick for traversal (refer to slide 29), if not already done. At this stage, all traversals start from the root node, without introducing additional optimizations (which you will get to below).

4. Extend your implementation by adding the phase-showstopper logic (refer to slide 30).

5. Modify your implementation to make Rule 1 extensions implicit using the global-end trick (refer to slides 31–33) when a leaf node is created via Rule 2. Also implement the Last_{j_i} index (refer to slides 32–33). However, even up to here, continue to perform explicit suffix extensions starting from the root node.

6. Enhance your code to create suffix links whenever a new internal node is formed via Rule 2 (regular case) (refer to slides 22–26).

7. Finally, optimize traversal by avoiding repeated starts from the root. Use the concepts of the active node, suffix links, and the remainder substring to efficiently perform subsequent suffix extensions (refer to slides 24–27).

```
-=0=-
 END
-=0=-
```